

Single Responsibility Principle C++



explain single responsibility principle and give examples in c++

The **Single Responsibility Principle (SRP)** is one of the five **SOLID** principles of object-oriented programming. It states:

A class should have only one reason to change, meaning it should have only one job or responsibility.

In simpler terms, a class should focus on **doing one thing well**, and **changes** to the class should only come from **one part** of the software.

💡 Why SRP Matters

When a class has more than one responsibility:

- It becomes harder to maintain.
- Changes in one part of the system might affect unrelated parts.
- It's harder to reuse and test.

✅ Example of SRP in C++

Let's say we're building a simple report system.

❌ Violating SRP

```
cpp

#include <iostream>
#include <string>
using namespace std;

class Report {
public:
    string title;
    string content;

    Report(const string& t, const string& c) : title(t), content(c) {}

    void print() {
        cout << "Title: " << title << endl;
        cout << "Content: " << content << endl;
    }

    void saveToFile(const string& filename) {
        // Imagine saving to a file here...
        cout << "Saving report to " << filename << endl;
    }
}
```

```
};
```

Here, the `Report` class does **too much**:

- It holds the data.
- It prints the data.
- It saves the data to a file.

That's **three responsibilities**.

✓ Applying SRP

Split the responsibilities into separate classes:

```
cpp

#include <iostream>
#include <string>
using namespace std;

// 1. Holds data
class Report {
public:
    string title;
    string content;

    Report(const string& t, const string& c) : title(t), content(c) {}
};

// 2. Handles printing
class ReportPrinter {
public:
    void print(const Report& report) {
        cout << "Title: " << report.title << endl;
        cout << "Content: " << report.content << endl;
    }
};

// 3. Handles file saving
class ReportSaver {
public:
    void saveToFile(const Report& report, const string& filename) {
        // Imagine file saving here...
        cout << "Saving report '" << report.title << "' to " << filename << endl;
    }
};
```

✓ Usage

```
cpp

int main() {
    Report report("SRP Example", "This report explains the Single Responsibility Principle.");

    ReportPrinter printer;
    printer.print(report);

    ReportSaver saver;
    saver.saveToFile(report, "report.txt");

    return 0;
}
```

```
}  
,
```

Summary

Class	Responsibility
<code>`Report`</code>	Holds data
<code>`ReportPrinter`</code>	Prints the report
<code>`ReportSaver`</code>	Saves the report to a file

This makes your code **easier to test, maintain, and scale**.

Would you like an example using inheritance or interfaces next?



yes

Awesome — let's explore how the **Single Responsibility Principle** works with **inheritance** and **interfaces (abstract classes)** in C++. This is especially helpful when you want different implementations for certain responsibilities — like printing to screen, PDF, or file.

Goal: Separate Responsibilities using Interfaces & Inheritance

Let's enhance our ``ReportPrinter`` class from earlier, using an **abstract base class** so we can support **multiple output formats**.

Step 1: Define the Base Interface

```
cpp  
  
`class IReportPrinter {  
public:  
    virtual void print(const Report& report) = 0; // Pure virtual  
    virtual ~IReportPrinter() = default;  
};
```

Now, any class that prints a report **must implement this interface**, which focuses only on the "printing" responsibility.

Step 2: Implement Different Printers

```
cpp  
  
`class ConsoleReportPrinter : public IReportPrinter {  
public:  
    void print(const Report& report) override {  
        cout << "[Console Output]" << endl;  
        cout << "Title: " << report.title << endl;  
        cout << "Content: " << report.content << endl;  
    }
```

```
};

class HtmlReportPrinter : public IReportPrinter {
public:
    void print(const Report& report) override {
        cout << "[HTML Output]" << endl;
        cout << "<h1>" << report.title << "</h1>" << endl;
        cout << "<p>" << report.content << "</p>" << endl;
    }
};
```

Each class does **one job** and adheres to **SRP**.

Step 3: Use in Main

```
cpp

`int main() {
    Report report("SRP with Interfaces", "This example uses interfaces for printers.");

    IReportPrinter* printer = new ConsoleReportPrinter();
    printer->print(report);
    delete printer;

    printer = new HtmlReportPrinter();
    printer->print(report);
    delete printer;

    return 0;
}
```

Benefits

- Each printer class has a **single reason to change**.
- You can add more formats (like `PdfReportPrinter``) without touching the core logic.
- You follow **Open/Closed Principle** too (open for extension, closed for modification).

Would you like to continue this with a `ReportSaver`` interface (for saving to file, DB, cloud, etc.) or integrate dependency injection?



integrate dependency injection

Nice — let's integrate **Dependency Injection (DI)** into our SRP example! DI is a powerful technique that helps make code **loosely coupled**, **easier to test**, and fully aligned with SRP and other SOLID principles like the **Dependency Inversion Principle (DIP)**.

What is Dependency Injection?

Instead of a class **creating** its own dependencies, they're **injected** from the outside — either via constructor, setter, or method injection.

Let's use **constructor injection** here.

Step 1: Our Interface and Implementations (from before)

Interface

```
cpp

`class IReportPrinter {
public:
    virtual void print(const Report& report) = 0;
    virtual ~IReportPrinter() = default;
};
```

Implementations

```
cpp

`class ConsoleReportPrinter : public IReportPrinter {
public:
    void print(const Report& report) override {
        cout << "[Console]" << endl;
        cout << "Title: " << report.title << "\nContent: " << report.content << endl;
    }
};

class HtmlReportPrinter : public IReportPrinter {
public:
    void print(const Report& report) override {
        cout << "[HTML]" << endl;
        cout << "<h1>" << report.title << "</h1><p>" << report.content << "</p>" << endl;
    }
};
```

Step 2: Class that Uses the Printer (Injected)

We create a class responsible for **rendering or managing report output**, but it **does not care** how it's printed — just that it **can be printed**.

```
cpp

`class ReportManager {
private:
    IReportPrinter* printer; // dependency

public:
    // Constructor Injection
    ReportManager(IReportPrinter* printer) : printer(printer) {}

    void displayReport(const Report& report) {
        printer->print(report); // uses the injected printer
    }
};
```

Step 3: Using It in `main()`

```
cpp

`int` main() {
    Report report("Dependency Injection", "This example uses constructor injection.");

    // Choose the printer at runtime (flexible!)
    ConsoleReportPrinter consolePrinter;
    HtmlReportPrinter htmlPrinter;

    // Inject Console printer
    ReportManager manager1(&consolePrinter);
    manager1.displayReport(report);

    // Inject HTML printer
    ReportManager manager2(&htmlPrinter);
    manager2.displayReport(report);

    return 0;
}
```

Why This is Cool

- `ReportManager` has **only one reason to change**: if the logic of how it handles reports changes.
- It doesn't care **how** printing happens — that's handled by injected classes.
- You can **easily swap implementations** for testing, extending, or reusing.

Next Level Ideas

Want to go further?

- Add a `ReportSaver` interface and inject it the same way.
- Use `std::unique\_ptr<IReportPrinter>` for ownership management.
- Use a DI framework (like [Boost.DI](#)) for auto-wiring dependencies.
- Add mock implementations for unit testing.

Let me know if you want help with any of those!